ECE2072 Project

Welcome and Getting Started

Welcome to the ECE2072 project, where you will be creating a processor that can run a set of given instructions. Your project to do so has been broken up into 4 main sections:

- Task 1: Create and test the building blocks required to make the processor
- Task 2: Create and test a processor that implements a handful of basic instructions
- Task 3: Extend your processor to implement extra, more complicated instructions.
- Task 4: Use instruction memory and a branch instruction to run programs.
- Report: Explain and justify the design decisions that were made during the creation of this processor

This document is long (although only 18 pages actually contain things you have to do). To get started with this project, we recommend you:

1. Read the first 5 pages of this document, which introduce the project and logistics
2. Do a ‘high level’ (skim) read through of the specification of the x72 processor first (on Moodle)
3. Do a ‘high level’ read through this document
4. Compare notes and understanding with your partner
5. Start attempting task 1

You should frequently come back to the detailed specification as required to ensure your design meets all of the requirements (this is to give you experience with implementing complex requirements).

At the bottom of the document, you will also find:

- A marking guide to walk you through how your project will be marked.
- A report guide to walk you through our expectations of your report.
- A FAQ section to answer any common questions that people have. This will be updated as time goes on to answer any recurring questions.

Table of Contents

Introduction and Learning Outcomes

This project builds on the knowledge you have developed, and gives you an opportunity to demonstrate what you have learned around the digital design process. Your solution is expected to be written in **high-level Verilog code** and make use of Verilog language features that you have been introduced to in the labs. You will make use of and solidify your understanding of:

- Behavioural Verilog Code
- Finite State Machines
- Test Benches
- Digital Logic Design
- Sequential Logic

You are also required to write a formal report that explains and justifies your design process and decisions.

We do not expect you to be familiar with processors, datapaths, or any related concepts. To get you up to speed, and introduce the way the x72 processor itself works, we have compiled a lesson about processors and datapaths for your convenience (on Moodle here). Depending on your background, you may also need to do some independent reading about the basics of processors and datapaths (and cite your references) in order to fill in any gaps you may have, although this should not be necessary to complete the majority of the project.

Revision History

Use this revision history to monitor for any changes we may need to make to this document to clarify requirements or provide hints

Description	Version	Date
Initial release	1.0	4/9
Added note to task 2 description about R0-R7 ports on processor	1.0.1	08/09
Updated lesson step by step break down of ssi instruction	1.0.2	11/09

Key Information and Notices:

The HDL Style Guide you are expected to follow for this project is available in appendix B.

Estimated Time Commitment: ~15-20 hours each for a Credit. A fully working solution and report will take considerably more time than this..

Groups:
The ECE2072 project is to be completed in **pairs**. You can nominate your preferred partner. If you do not nominate your preferred partner by the end of week 7, you will be assigned one. You may nominate someone from a different lab (subject to capacity), but you both must be able to attend the same lab session for marking in week 12.

If you wish to make alternate arrangements, please reach out to the teaching team.

Academic Integrity
You may discuss **general** details (eg: approaches) of the project with other students. However, **you may not:**

- write the code together with students from other groups,
- show student from other groups your HDL,
- copy other group's work, or
- present work you have found online as your own (i.e. without attribution),

This is plagiarism and/or collusion and is prohibited by Monash's academic integrity policy. Breaches of this policy attract serious consequences. Note that all code submitted to this unit will be run through the MOSS system to check for plagiarism.

If you research some concept online or re-use HDL from your labs (or from this unit) you must cite it (any citation method is acceptable).

Generative AI:
As a reminder, you can use generative artificial intelligence (AI) in order to troubleshoot your solutions, explain concepts and assist you in preparing your reports.
Both students in a group must **fully** understand the work submitted for assessment - if you do not understand your work, you may receive a zero interview score, which will result in a zero for the project.

Any use of generative AI for this purpose **must** be appropriately acknowledged, if you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines. You **must not use** generative artificial intelligence (AI) to generate any HDL for submission.

Submission, Presentation & Marking

Design Due: Friday, 11th October, 11:55 PM (End of Week 11)
Presentation: Assigned time slot in scheduled lab session (Week 12)
Report Due: Friday, 18th October, 11:55 PM (End of Week 12)

Submission File Structure:
You are required to submit your entire Quartus project on Moodle, along with a 1 page 'work summary' document, describing which parts of task 1 you've each done, and both of your overall contributions to the project, which we will use to have a discussion about scaling your individual scores if required. You can do this by archiving the project file via Quartus. A checklist is provided here.

We will be marking your functionality by importing your Quartus Archive, and compiling it on the lab machines. **Mark penalties will apply if your marker has to debug your design.**

Therefore, ensure you have tested your project submission by re-downloading your .qar file (after you have placed it on Moodle) and testing it for functionality BEFORE clicking submit. Ensure all relevant files are present in your .qar's file directory.

We will mark you on your choice of device (DE2-115/DE10).

You must submit your formal report in PDF format. You will be provided with a google drive folder for your team, which should include all 'artefacts' (i.e. documentation, test benches, outputs) you need to support your report. You should hyperlink from your report to the appropriate document you wish to reference to support your work (think of it like a searchable appendix). Testbench artefacts should be properly exported as per the instructions in Lab 2.

Late Submissions:
The design and the report are separate submissions. Late submissions will only be accepted until your lab class unless special consideration is granted. The standard (5%) penalty applies for every late day.

Special Consideration:
For extensions, you must apply through Monash Special Consideration. **The report is a separate assessment and must be extended separately.** Note if you receive an extension until after your scheduled lab class, you will need to attend an alternative session to demonstrate your work. **You must still demonstrate your work in person with your partner.**

Mark Distribution:
The 10% of the final project grade is broken down between the design task (7%) and the report (3%), with the whole project scaled by an individual interview. A detailed mark distribution is included at the end of the project, and a rubric will be linked from this section.

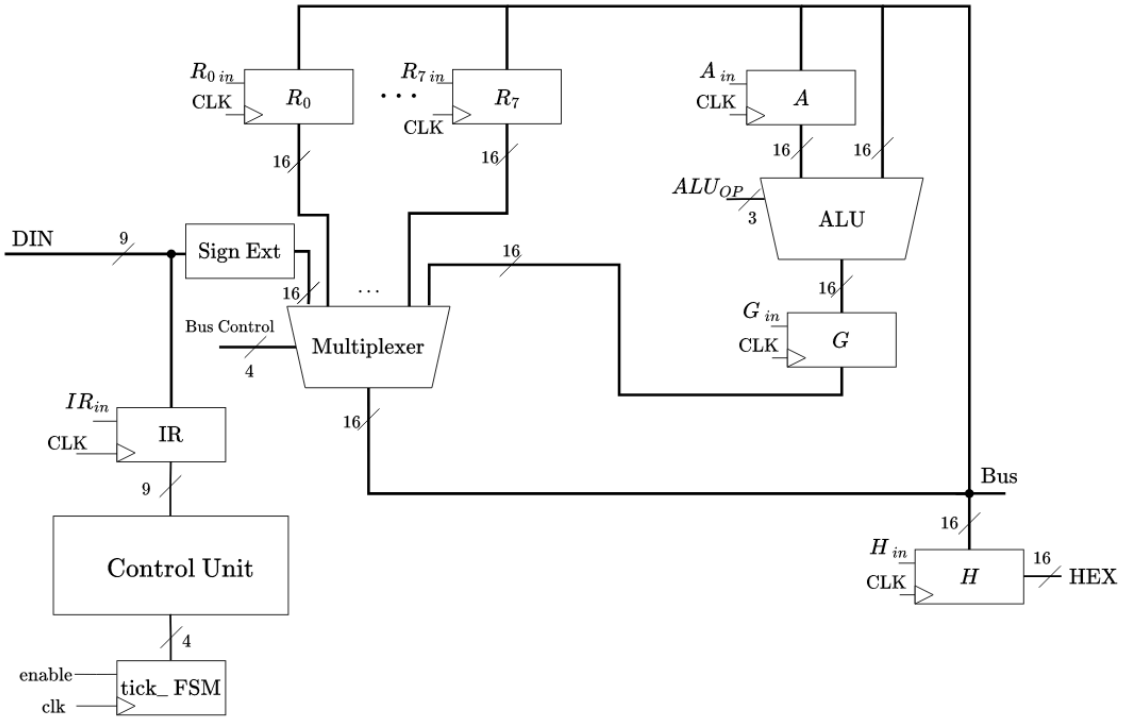
Completing tasks 1 and 2, coupled with a ~credit level report, will get a credit overall.

Background Overview

The following is an abridged version of the background information supplied in the processors and datapath moodle lesson, which introduces this architecture. We will refer back to this lesson for key details throughout this document.

A computer processor, also known as a central processing unit (CPU), is a sequential digital circuit that performs the calculations that make a computer 'run'. It performs arithmetic and logical operations, input/output (I/O), and other basic instructions. In this project, you will be making your own processor, which we are calling an 'x72'. You will begin by creating a simple processor, and progressively add additional features and functionalities.

A simple processor has several individual components that all work together to perform operations, the x72 processor that is designed in the project is shown in the image below.



Task 1: The CPU Components

To begin, we must first create each of the building blocks (components) we will use to make our processor. Using the provided skeleton file named `components.v`, write Verilog HDL to implement each of the 5 component modules we require to construct the simple CPU.

The specifications for the components you are required to create are described below. Note that you do **not** need to implement the control unit in task 1.

The design and implementation of these modules must be split between both members of your group. Each student will receive a score for task 1 based only on the functionality of the components they designed. This split must be formally documented in both your HDL (via comments) and in your report.

Note, you are each still required to completely understand the functionality of **all components.** This is important as they will be needed in every subsequent part of the project. You may be asked questions about **any** of these components during your interview.

1 - Sign extender:

You may have noticed that a 9 bit DIN input wire doesn't have the same width as our 16-bit data bus. You should write a sign extender module to deal with this issue. When a 9-bit word is inputted to this module, the output should be the corresponding sign-extended 16-bit binary number. To sign-extend, you need to pad the most significant bits [15:9] with a '1' if the input number is negative, and a '0' if positive. You should try this by hand to make sure that your module works for both positive and negative 2s complement number

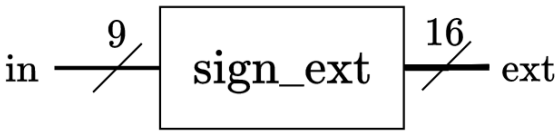


Figure 1: A high-level diagram of the sign extension module.

2 - Tick Counter:

Design and write a Tick finite state machine to maintain the current state of the processor. Your FSM must meet the following specifications. It should have three input (an enable, a reset, and a clock), and a 4 bit output to represent the 4 possible ticks of the system. When enable is asserted, the output should cycle through a state representing each of the four ticks described in the section "detailed breakdown of instructions" in the x72 lesson in Moodle.

Your FSM must make use of one-hot coding, and use the minimum number of both registers and states to implement the required functionality, given this constraint.

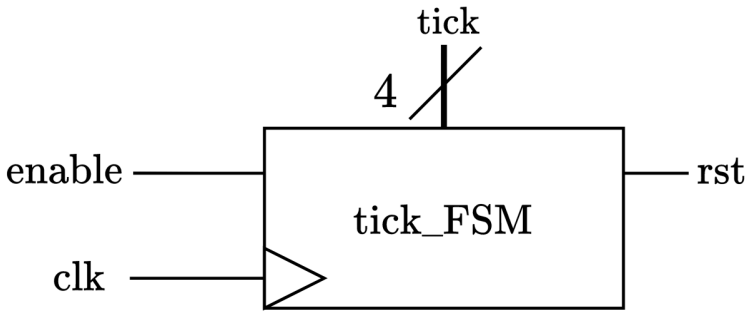


Figure 2: A high-level diagram of the tick_FSM module

In `components.v`, write a module `tick_FSM` that implements the FSM described above. A skeleton has been provided for you in the skeleton code in `components.v`. Note that you **must not** change the names of the input and output ports for the `tick_FSM` module (these need to match the image shown in Figure 2).

3 - The ALU:

The ALU module, as shown in figure 3 below, has three inputs and one output. The ALU performs operations on the two 16-bit inputs `input_a` and `input_b`, based on the select input of `alu_op`.

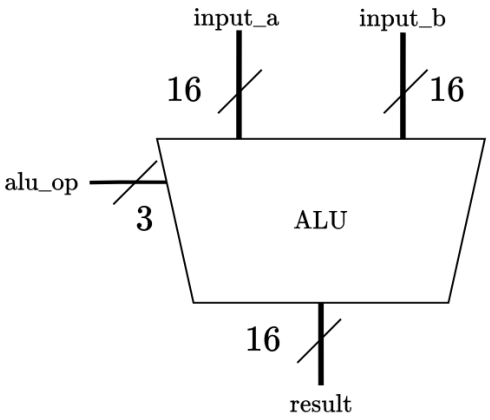


Figure 3: A high-level diagram of the ALU module

The operation to perform corresponding to the alu_op input is summarised in table 3 below.

Table 3: ALU operation summary.
input_a comes from register A, input_b comes from the bus wires

alu_op	Operation
000	input_a * input_b (multiplication)
001	input_a + input_b (addition)
010	input_a - input_b (subtraction)
011	input_b shifted by input_a (signed shift)
100	None (don’t care)
101	None (don’t care)
110	None (don’t care)
111	None (don’t care)

In `components.v`, write an ALU module named `alu` that implements the logic described above. A skeleton has been provided for you in the skeleton code in `components.v`. Note that you **must not** change the names of the input and output ports for the `alu` module.

4 - The Multiplexer:

The multiplexer module, as shown in Figure 4 below, is how the bus of the processor is implemented. It has ten 16-bit data inputs - for registers 0-7, register G, and the sign extended version of the 9-bit data input (DIN). It has a 4-bit input select line that controls which data input appears on the output.

The output of the MUX will be used to feed into registers 0 through 7, register A, register H, and the ALU. The values on the bus wires will only be stored into a register when the `enable` signal on that register is asserted.

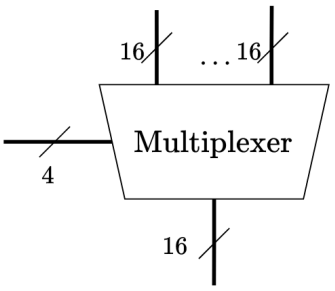


Figure 4: A high-level diagram of the multiplexer module

5 - Registers:

The register module, as shown in figure 5 below, has four inputs and one output. On the rising edge of the `clk` input and if the `rst` input is asserted, then the register contents is reset to 0. Otherwise, if the `r_in` enable input is asserted, then the data on the data input, `data_in`, is stored in the register. The data output, `Q`, is the value that is currently stored in the register.

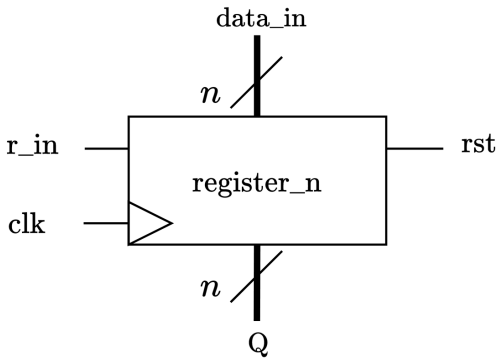


Figure 5: A high-level diagram of the register module

In `components.v`, write a register module named `register_n` that implements the logic above. Your design should use a parameter `n`[1] that allows the number of bits that the register stores to be changed across instantiations as the instruction register and general purpose register holds different numbers of bits. A skeleton has been provided for you in the skeleton code for `components.v`. Note that you **must not** change the names of the input and output ports for the `register_n` module.

[1] Defining a variable parameter within a module definition allows you to control the variable during the module's instantiation. The skeleton code will contain instructions on how to do this.

Testbench:

After implementing each of the components above, write a testbench in a file named `components_tb.v` that implements a process to **intelligently** test each of the modules you have created. A skeleton has been provided for you in the skeleton code for `components_tb.v`.

Each student must claim responsibility, via HDL comments and the 'work summary' document for at least **two** of these test cases, the total number of test cases, and the work distribution is a choice for you to justify as a pair.

You will be assessed (in the live demonstration and report) on whether:

- The testbench checks each component
- The testbench **intelligently** checks all functions of each component
- The testbench reports clearly that a component has passed
- The testbench report clearly shows what tests have failed in a manner that would aid debugging
    - If you do have failed tests, you will receive more marks for having the testbench clearly documented rather than trying to hide them
- A summary of the results is clearly printed at the conclusion of the testbench

Task 2: The simple processor

In this task, you will implement the x72 processor architecture using your components from Task 1. For now, you will only be implementing a subset of the total processor. A simplified diagram containing only the aspects of this task are shown below:

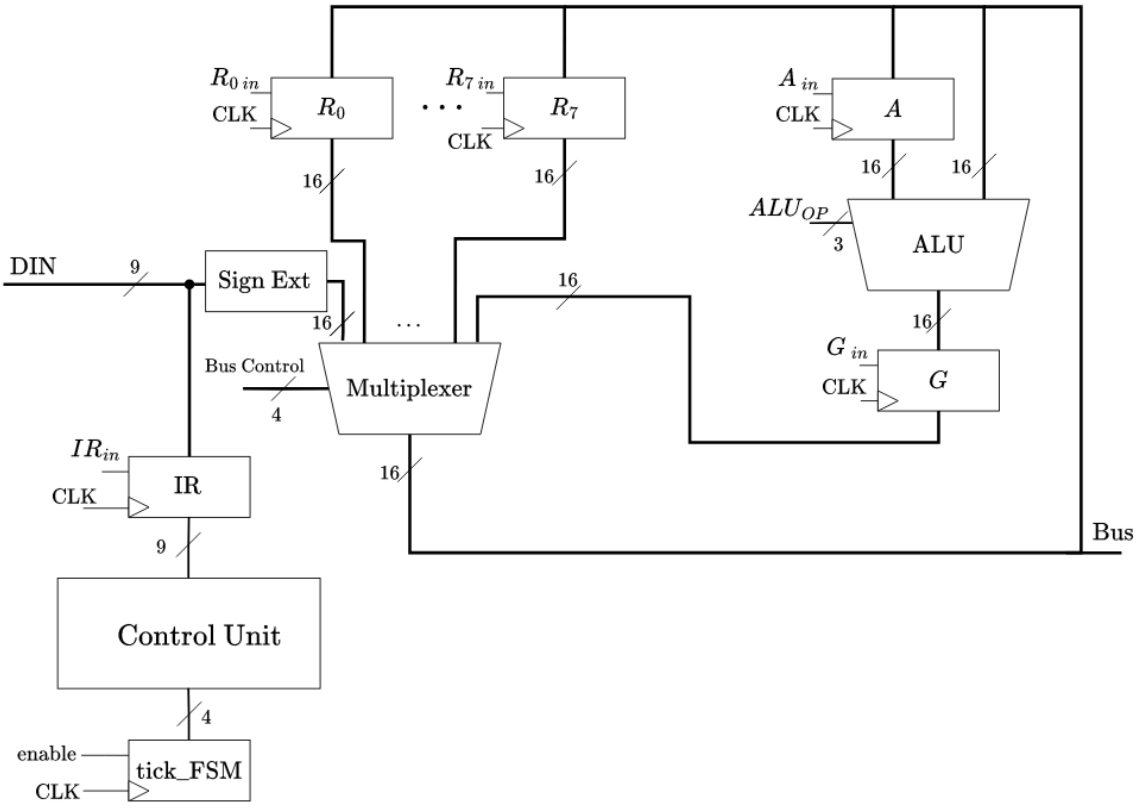


Figure 6: An internal diagram of the task 2 processor

You will implement the processor in a module named `simple_proc`. A skeleton has been provided for you in the skeleton code for `proc.v`. Note that you **must not** change the names of the input and output ports for the `simple_proc` module.

Note: The skeleton you are provided with includes output ports to output the values of the internal registers R0 - R7, for the purpose of test benching. When instantiating the processor to program your DE10-lite, you can leave these ports unused.

Your simple processor must support the following instructions (using the instruction encoding specified in the x72 lesson):

- Move immediate (movi)
- Addition (add)
- Add Immediate (addi)
- Subtraction (sub)

Testing your Implementation

Important: Functional marks for task 2 are for demonstrating that your system works. To do this, you must use a mixture of hardware and simulation. Your demonstration time is limited, and how you use it is part of the assessment. Therefore, note you will not be able to achieve full marks by trying to demonstrate full functionality via extended hardware simulation of many instructions (for hopefully obvious reasons).

Hardware in Loop Testing

To test with the DE10 (or DE2), write a top-level module which does the following:

- Instantiates the processor module
- Connects SW[8:0] to the bus input of the processor
- Connects ~KEY[0] to the reset input of the processor, to use as a synchronous reset
- Connects a 1'b1 to the enable input of the tick_FSM, to permanently 'lock' the processor into its ON state for this phase of the project
- Connects ~KEY[1] to the clock input of the processor
- Connects LEDR[9:0] to the bottom 10 bits of the bus output of the processor
- Decodes the tick_FSM output of the processor into a 7-seg format (e.g., using modified BCD) and displays the current tick on HEX5[6:0]

A block diagram of your top-level Verilog module should match this diagram:

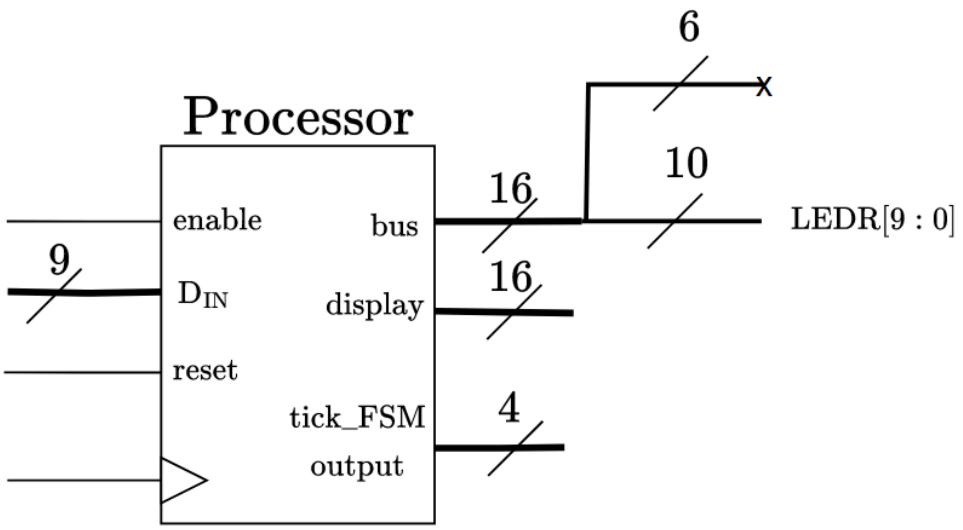


Figure 7: A block diagram of the top level module for task 2

You can then test that your processor works correctly by setting SW[8:0] to an encoded instruction, and pressing KEY[0] to pulse the clock. You can observe the values appearing on the bus (LEDR) to confirm the processor is behaving as expected.

Hardware testing is critical, but this is limited, and doesn't let us test multiple instructions or verify things easily. To fully test our processor, we will need to use a testbench.

In a file named `proc_tb.v`, write a testbench that instantiates the processor and simulates the execution of the specified instructions. You can view the waveform of the internal processor registers/wires to verify that your processor is correct, however this will not let you effectively test/prove accuracy of the instructions. Therefore, you must include print statements for your testbench.

For full marks, you must clearly demonstrate that your design meets the specifications of task 2. How you set up your "test campaign" to do this (i.e., the test benches you run) is part of the project, but must contain a mix of hardware and simulation results.

More specifically, you will be assessed (in your demonstration and report) on:

- What combination of cases and/or selection method you choose to use for your tests.
- The clarity and presentation of the reporting of your testbenches. This includes how you report errors/failed tests, even if they are not triggered.
- How you decide to operate your tests (ie: assumptions and configurations).
- Your justification of why your test cases are sufficient to prove correct operation.

There is no "one correct answer" to this problem. This is representative of many real world engineering scenarios you will run into as the years go by, so we believe it's important you get used to working with this type of open-ended challenge early in your degree.

Task 3: Adding an Output Peripheral and New Instructions

In this task, you will be adding an output peripheral to your processor in the form of a hex display. In a file named `proc_extension.v`, copy in the code from task 2 and modify the processor module to include a new output port named `display`, and support a `disp Rx` instruction, which should output the value of Rx to this new output `display` port.

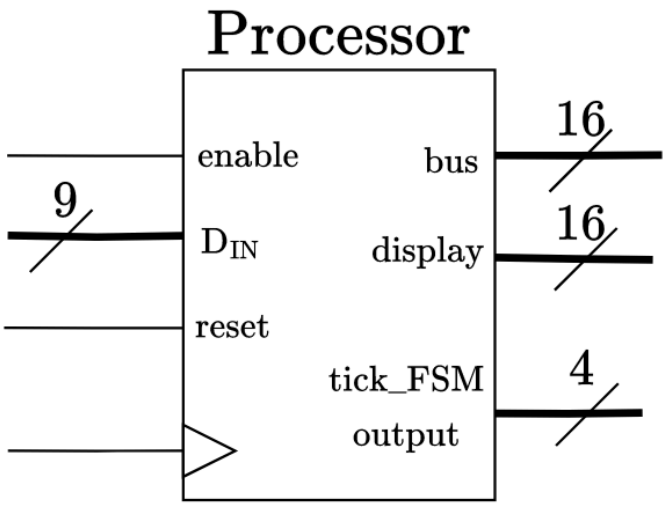


Figure 8: A high level diagram of the processor with a display output

Any value written to the display port should **remain displayed** while other instructions are executing, until the next `display` instruction is executed.

You can implement the `display` instruction by modifying the internal processor to include a new 16-bit 'display' register (also referred to as register H in the x72 architecture) that is connected to the output, and modifying the control unit to assert the $\texttt{display}_{\texttt{in}}$ control signal when the desired display value is output onto the bus.

Additionally, you will be adding support for the following instructions (using the instruction encoding specified in the x72 lesson):

- Signed-shift-immediate (`ssi`)[2]
- Multiplication (`mult`)
- Display (`disp`)

[2] For more information on the `ssi` instruction, see the x72 lesson.

Testing your Implementation

Important: Functional marks for task 3 are for demonstrating that your system works. It is up to you to decide how to do this. Your demonstration time is limited, and how you use it is part of the assessment. We recommend selecting a mixture of hardware demos and simulation results that demonstrate the extensions to your processor (on top of the functionality in task 2) work as intended.

Hardware Testing

Once you have implemented the changes to your processor, write a top-level module that does the following:

- Instantiates the processor module
- Connects SW[9] to the enable input of the processor
- Connects SW[8:0] to the DIN input of the processor
- Connects ~KEY[0] to the reset input of the processor, to use as a synchronous reset
- Connects ~KEY[1] to the clock input of the processor
- Connects LEDR[9:0] to the bottom 10 bits of the bus output of the processor
- Uses a BCD decoder to decode the display output of the processor to **HEX4** to **HEX0** as an **signed**[3] 16-bit integer.
- Decodes the tick_FSM output of the processor into a 7-seg format (e.g., using modified BCD) and displays the current tick on HEX5[6:0]

Your top-level module should look like the following:

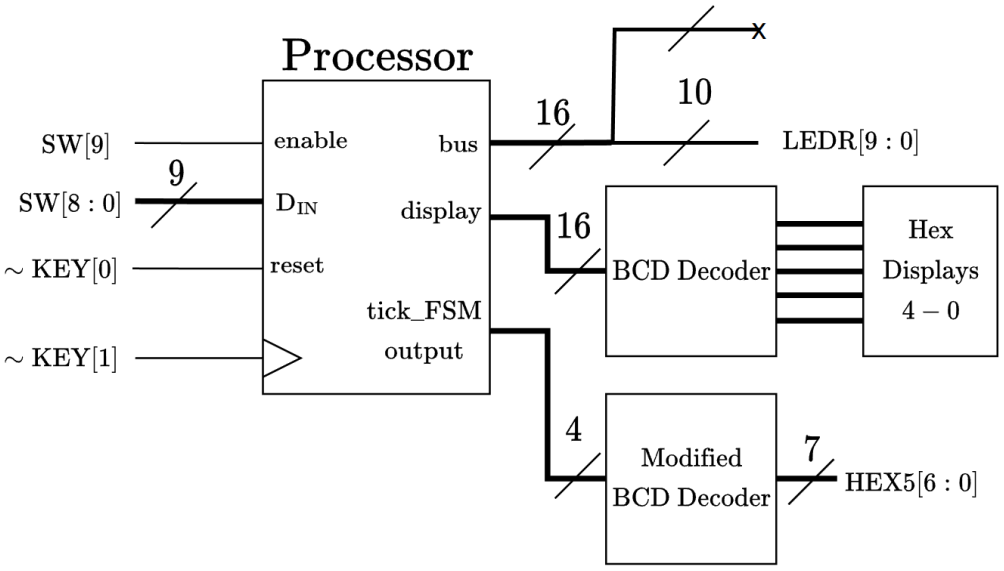


Figure 9: A block diagram of the top level module for task 3. Note we only want to extract the lower 10 bits from the register for the LEDR output.

[3] We don't have space to add a "-" sign, while also maintaining enough digits to displace a signed 16-bit integer. Therefore, you should use all of the decimal place outputs (the 8th wire of each HEX display) to denote when the decimal number displayed on HEX4 to HEX0 is negative.

Task 4: Instruction Memory and Branching

In this task, you will be interfacing your processor with some instruction memory made with a read-only memory (ROM) Quartus IP block, and extending your processor such that it is Turing complete.

This task is designed to be an extension for the students aiming for high-HD marks (85+%). We have not provided you with a template datapath to use. You may need to make several adjustments/additions to other components in your processor to successfully accomplish this task (new ALU instructions, extending the ticks per instruction etc).

Instruction Memory:

In a file called 'proc_memory', copy your Verilog from task 3, and add the program counter (PC) register to your processor's datapath to track where in memory your processor will take its next instruction from.

Next, create an instance of the Quartus ROM IP block that will be used as your instruction memory. Each word should be the same width as Din, and the block's size should represent the total amount of instructions that could be addressed by a 16-bit address. You should assume all instructions use 2 words for ease of use. See Appendix C for instructions on how to pre-program the ROM with assembly code.

In order to interface with the ROM IP block, your processor will need to export the current 16-bit program counter value from the new 'PC' output wires. This will then act as the address read in by the instruction memory, which will feed the requested instruction into the Din of your processor. At the end of each instruction, you will perform a '+2' operation on the value in the program counter (1 for the instruction, 1 for the immediate), so that the next instruction may be read in.

For immediate instructions, you would add 1 to the PC to retrieve the immediate value, then add 1 again for the next instruction.

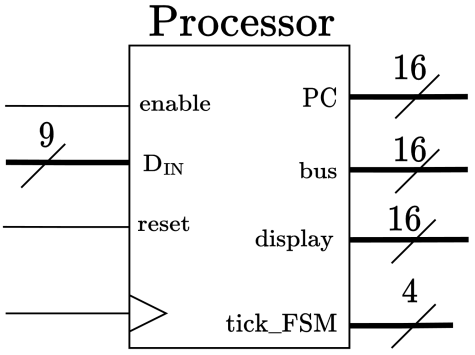


Figure 10: A block diagram for the processor module in task 4.

Branching:

Finally, you will add support for the `bez Rx imm` instruction which will adjust the current program counter value based on the value in `Rx` and the immediate provided (as specified in the x72 lesson in Moodle). You may assume that whoever wrote the assembly has correctly

encoded the instruction such that the value received contains the appropriate adjustments to the immediate value such that it represents the number of instructions that the PC will jump by. I.e. `bez 0, 6` is a 6-instruction jump.

Testing your Implementation

Once you have implemented the changes to your processor, write a top-level module that does the following (note, a testbench is **not** required for task 4):

- Instantiates the processor module (as specified in the x72 lesson)
- Instantiates the instruction memory as your ROM IP-block (See programming instructions in Appendix C)
- Connects the input address of the instruction memory to the processor's PC output.
- Connects the output of the instruction memory to the Din input of the processor
- Connects SW[9] to the enable input of the processor, so we can pause the program.
- Connects ~KEY[0] to the reset input such that we can fully reset the processor.
- Connects the clock input of the processor to a 10 Hz clock (see Lab 3 for instructions on making this yourself.)
- Connects LEDR[9:0] to the bottom 10 bits of the bus output of the processor
- Uses a BCD decoder to decode the display output of the processor to **HEX4** to **HEX0** as an unsigned 16-bit integer.
- Decodes the tick_FSM output of the processor into a 7-seg format (e.g., using modified BCD) and displays the current tick on HEX5[6:0]

Your top-level module should look like the following:

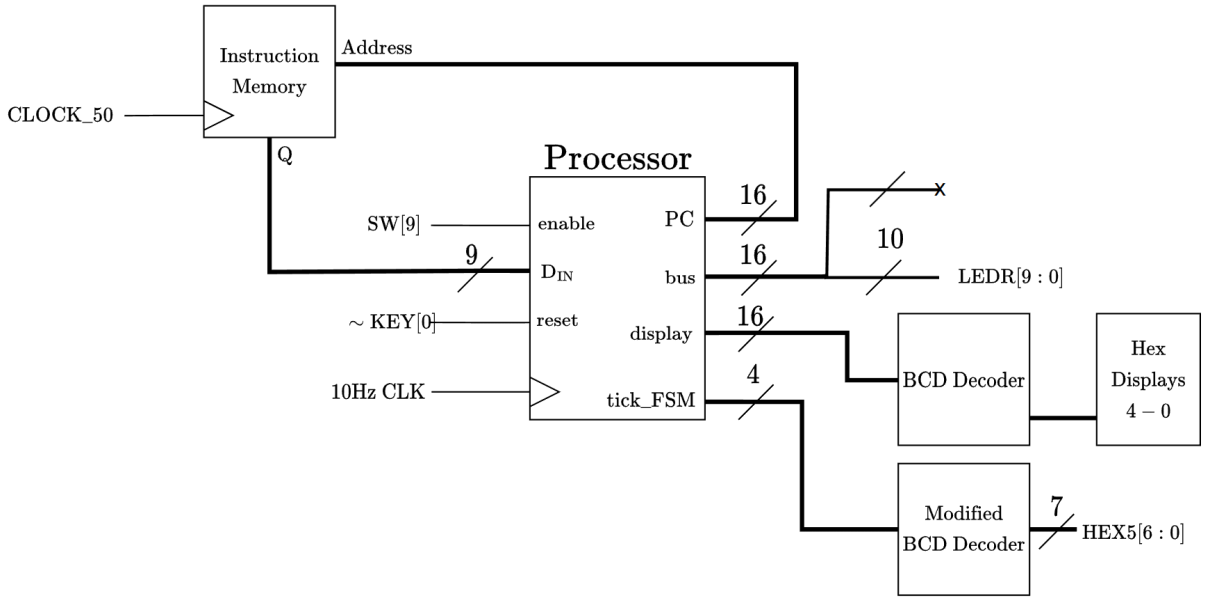


Figure 11: A block diagram of the top level module for task 4.

Reporting

We are expecting you to submit a formal design report, of approximately 6-8 pages (roughly 2500 words) not including references or appendices. Your report should contain:

- A brief introduction to what you were asked to do, and your design, assuming the person reading your report is not familiar with the x72 architecture or this document, but has completed ECE2072.
- An overview of each component you have implemented in task 1, highlighting any design choices you have made, the functionality of the components, and a summary of resource usage and maximum frequency of each component[4] (timing analysis).
- A detailed description of how your implemented datapath executes two instructions (one after the other) of your choice. Choose your instructions to best illustrate the functionality you have achieved (if you have only done task 2, do ones from task 2)
- A detailed discussion of the design of your testing regime for all tasks (at least 3 pages), including summarised key results (including waveform excerpts if appropriate) from the testbenches.
- A conclusion, summarising your justification that your design is functional and fit for deployment.

We realise that 6-8 pages isn't a huge amount of space for this much information! Part of writing well technically is learning how to be concise, which is also part of your challenge with this project.

Going over this amount of pages/words will not result in mark deductions **provided** you have good reason to do so. If you could have been more concise, you will lose marks. Playing around with formatting to 'squeeze in' more text will not make your report clearer, and so will also most likely attract a deduction.

You can find further details on the standards and requirements of technical reports from the **Monash Report Standards** site.

[4] For more information on completing the timing analysis, see Appendix A

Mark Distribution

Design (70 total, for 7%)

Task 1 Function/Demonstration: The CPU Components (individual)

Detailed presentation of design and implementation of components	5
Testbench outputs are correct with a summarised output	5
Total for Task 1:	10

Task 2 Function/Demonstration: A Simple Processor

Live hardware demonstration of at least 2 instructions[5]	10
Live hardware bus and tick_FSM display appropriate values	5
Detailed discussion justifying the design of your testbench	5
Live demonstration of testbench operation	5
Presentation of testbench results justifying processor validity	5
Total for Task 2:	30

Task 3 Function/Demonstration: Output Peripherals & New Instructions[6]

Live hardware demonstration of disp function (HEX displays)	5
Detailed discussion justifying the design of your testbench	5
Presentation of testbench results justifying new processor validity	5
Total for Task 3:	15

Task 4 Function/Demonstration: Memory & Branching

Detailed discussion on design choices made for task 4	5
Demonstration of an assembly program that makes use of all features	10
Total for Task 4:	15

Deductions:

Design does not adhere to HDL Style Guide and proper coding standards & practices per item missing from list (capped at -10)	-1
Warnings are output when compiling all programs	-10
Pair does not include 1 page summary of work completed with design	-5

[5] You may present any instruction here if you have also completed task 3 and/or 4.
[6] This task may be presented together with task 2 if you have done both.

Reporting (30 total, for 3%)

Introduction	2
Component Overview, Functionality and Design Choices (Task 1)	3
Resource Usage and Timing Analysis (Task 1)	2
Two Instruction Datapath Implementation Example (Task 2/3)	6
Design of Testing Regime (All Tasks)	9
Conclusion and Justification	2
Formatting and Writing	4
All required information included in appendices	2
Total for Report:	30

Individual Interview (Scaling)

Category Description	**Detailed Description**	**Scaling Factor**
Complete Understanding	The student has **clearly prepared** and **understands the design**. They can answer the questions correctly and concisely with little to no prompting.	100%
Tolerable understanding	The student is **reasonably well prepared** and can **consistently provide answers** that are **mostly correct**, possibly with some prompting. The student may lack confidence or speed in answering.	90%
Selective understanding	The student gives answers that are **partially correct or can answer questions about one area correctly** but another not at all. The student has not prepared sufficiently.	80%
Trivial understanding	The student may have seen the design/circuits before, and can answer **something partially relevant or correct** to a question but they **clearly can't engage in a serious discussion of the work**.	70%
No Understanding / Absent	The student has **not prepared**, **cannot answer even the most basic questions** and likely has not even seen the design before, or did not attend the interview.	0%

Notes:

- Submitting code that does not compile/synthesise will result in a **heavy deduction**.
- Live demonstration requires you to show an understanding of your implementation by answering questions verbally during the demonstration. If you do not understand your submitted code during the interview, your project score may be scaled as low as 0.
- A peer assessment scaling may be applied to your work.
- A detailed breakdown of the rubric will be released shortly.

Submission checklist

There are 2 submissions for this project, the Design and the Report.
For the **Design:**

- Your .qar file to the “Project - Design (.qar) Dropbox” which should contain:
  - Verilog code for tasks 1, 2, 3, and 4
  - Testbench codes for tasks 1,2, 3 and 4
  - .sof files for all the tasks you have completed
  - Your .qar file should be named Project_GroupXX.qar (eg Project_A88.qar)
- A 1 page summary of the tasks you have completed, and intend to demonstrate, using this template. This should include:
  - A table allocating the work distribution between the two of you (eg 50/50 if equal)
  - A table summarising the individual work allocation and responsibilities as specified in task 1
  - What you intend to demonstrate
  - Notes about anything that is not functioning, if applicable

For the **Report:**

- Your PDF report to the “Project - Report Dropbox” which should contain:
  - All points outlined in the Reporting section
  - Your PDF file should be named Report_GROUPXX.pdf (eg Report_A88.pdf)
- All artefacts utilised to justify your work in the provided gdrive folder “Project_Artefacts_Group_XXX”. This includes:
  - Verilog code for tasks 1, 2, 3, and 4
  - Testbench codes for tasks 1, 2, 3, and 4
  - Relevant selection of ModelSim outputs or waveforms (as text or screenshots) for each task, as needed to demonstrate your points

Live demonstration procedure

After you submit your project at the end of week 11, you will be required to demonstrate your project live during your assigned lab session in week 12. You will book a timeslot within your lab session to do this.

In this timeslot, your demonstrator will download your submission from Moodle, and compile your design from the qar file. You will then proceed to demonstrate your working solution to the final task you completed (for example, if you have done all 3 tasks, we will ask you to show us instructions from task 2 running the full hardware), this will help you ‘tick off’ the marking items efficiently .

You will only have 10 minutes to demonstrate your working solution to the TAs. You will need to become proficient with discussing and demonstrating the functionality of your processor using both the hardware and the testbenches. After the demonstration process, the TAs may need to ask you questions regarding the project to complete the scaling interview, if this was not completed during the demonstration.

FAQ

This section will be updated live over the rest of the semester as we receive more questions.

1. How do I submit my project?
    a. You need to follow the submission instructions at the start of the project
2. What does “Adherence to proper Verilog practice” mean?
    a. This means we expect you to follow proper programming practices for Verilog, such as no inferred latches, no redundant defined bits etc.
3. How strict is the word/page limit?
    a. Going over this amount of pages/words will not result in mark deductions **provided** you have good reasons to do so. If we think you could have been more concise, you will lose marks.
    b. Playing around with formatting to ‘squeeze in’ more text will not make your report clearer, and so will also most likely attract a deduction, so please don’t try that either.
4. What is meant by “intelligent testing”?
    a. The short answer is this is really up for you to justify to us in your report and your presentation to us in week 12. As long as you can justify that whatever your tests are are effectively testing (using the criteria I have given in the testing sections of tasks 1*, 2, and 3) the design, you'll get your marks. There are many approaches, you've identified at least 4 options, there are others you could research, and for example, I covered one in workshops/readings - using an LFSR.
5. I was just wondering if we can use megafunctions to help create some of the components. Would this be a good approach or will the resource usage make it a problem in the long term?
    a. That is fine as long as you can justify it. As you mentioned, you might want to write up a few components and test the resource usage with the mega functions first.
6. We have a lot of components in Task 1. Do we have to do all the tests in components_tb.v? Can we have, for instance, alu_tb.v for alu, tick_tb.v for tick_FSM etc?
    a. Yes you can split it up when you're trying to prototype each component, but for your final submission, you need to only submit two files (components.v and your components_tb.v) for this task
7. The proc.v module seems to be missing an output for the FSM_tick, can we add that in?
    a. Yes, you can add in the FSM_tick as an output.
8. Can we assume that if the input was a type 1 instruction, then the din will not be changed for the duration of the 4 tick cycle?
    a. This one might be difficult to justify, so I would recommend considering your implementation of how din would affect the processor in each FSM_tick.
9. In the simple_proc module, R0 to R7 are internal registers and are not set directly by an external input so why are they listed as input ports?
    a. They can be used for testbenching so you can see how the register values changes.
10. Can the tick output on HEX5 be 1 (0001), 2 (0010), 4 (0100) and 8 (1000), instead of 1, 2, 3, 4. Since I can't find it specified it just says that it needs to show what tick it is

    a. Sure, this just seems like something you might need to justify in your reporting
11. When making the ALU what are we meant to do with the bit overflow when adding multiplying or left bit shifting, are we to disregard it and just output the first 16 bits?
    a. Up to you to justify what you think is a reasonable approach. ie. any choice that meets the published specs is fine, but without justification on why you made it, you will lose marks in your reporting
12. What does “Resource usage and timing analysis of each component is documented clearly and provides clear insight of the result” mean?
    a. It needs to have an accompanied discussion outlining your understanding of the resource usage and timing analysis
13. When I try to run my testbench for task1, Modelsim cannot find the lpm_mult file needed (I'm using the multiplier IP block for the ALU), even though it compiled correctly
    a. In the menu, go to Simulate, then after selecting the file you want to simulate, in the Libraries tab select lpm_ver.
14. What referencing style should we use?
    a. IEEE preferred, but if you chose another one and stayed consistent, then that is fine too
15. I cannot get rid of some of my warnings, what should I do?
    a. You can justify the presence of the remaining warnings
16. My testbench loads modelsim but does not load my design when I drag it into “wave”, what do I do?
    a. If your testbench loads modelsim, then you have (hopefully) set the correct top level file (the testbench file). The resulting error is from syntax/semantic errors from your code which prevents your modelsim from loading the wires/registers into the simulator. To debug, we usually recommend 'cutting down' what you are testing from the testbench until you have something that works, then adding work back piece by piece. There may also be error/warning messages within modelsim itself (it is written in blue)
17. Do we need to reference the ECE2072 Project (this document)?
    a. No, you do not need to ref the project. You can cite the unit materials if you need to, eg, https://ieee-dataport.org/sites/default/files/analysis/27/IEEE%20Citation%20Guidelines.pdf (linked from Monash lib)

Appendix A: Timing Analysis

When completing the timing analysis for each portion of the lab, you can simply re-use the `analysis.v` file that we provided in lab 2. You will need to change the width of the inputs and outputs. You will also need to make sure that there is a register on each end of the module under test's inputs and outputs (where they exist).

Appendix B: HDL Style Guide

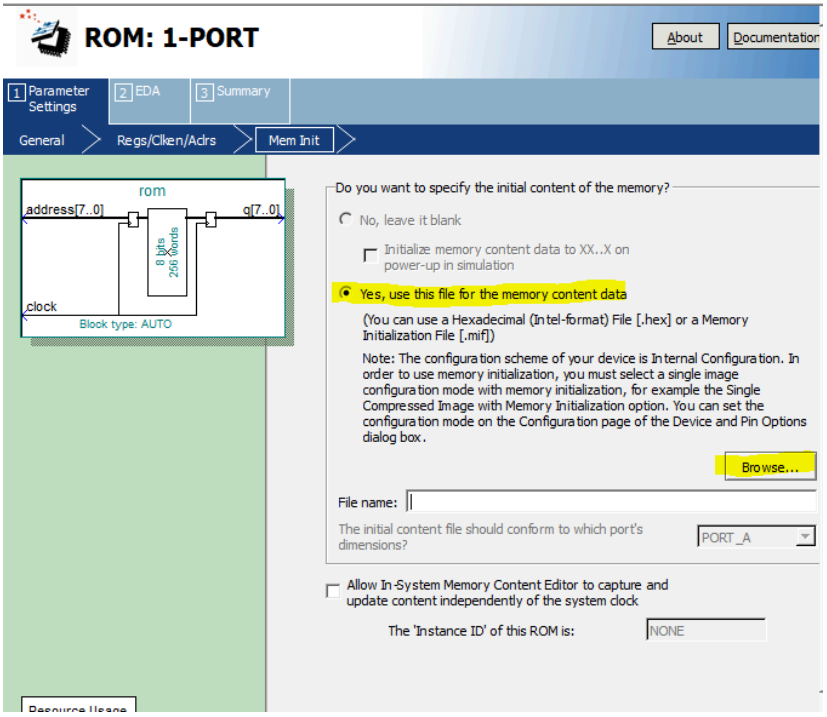
When writing Verilog, you are expected to follow these guidelines:

1. Use **[N-1:0] ordering** of multi-bit signals unless there is a good reason to use unconventional ordering.
2. **Non blocking** assignments <= should be used in posedge clock always blocks. Each bit in <= assignment can synthesise a flip-flop.
3. Do **not mix non-blocking and blocking** assignments in the same always block.
4. Do **not drive the same signal in two different blocks** within the same module.
5. **Cover all input combinations** for defining combinational logic from always blocks. A default assignment at the top of the block is a good way to do this.
6. Think about whether **don't cares** should be used in the default case statements.
7. **Use synchronous resets** unless there is a good reason to violate timing constraints with asynchronous resets.
8. **Synchronise all asynchronous inputs** to the system clock with two cascaded flip-flops to allow for metastable settling.
9. **Use just one global clock** with *posedge* triggering in your designs. Multiple clocks and posedge with negedge triggering can cause timing problems. Clock enables (CE) can be used to select lower rate sampling than the global clock.
10. Ensure you have **enough bits** on signals that are connected within an instantiation. The compiler will issue a warning message if insufficient bits are provided.
11. Use **named associations** in port mapping for instantiations with more than 2 ports, e.g.:
    ```
    Dflipflop dff1( .iClk(clk),
    .iRst(rst),
    .iD(data),
    .oQ(out)
    );
    ```
    rather than relying on order:
    ```
    DFlipflop dff1(clk, rst, data, out);
    ```
12. Check all warning messages after compiling in Quartus and ModelSim. See 10.
13. Use all uppercase for PARAMETER_VALUES, and lowercase for signals.
14. Avoid pointless parameters such as ONE, TWO, THREE for 1, 2, 3.
15. Beware of using operators % / * (modulus divide and multiply) because they synthesise to large circuits!
16. **Check your synthesised circuit** after compiling. See Quartus resource usage and Quartus:Tools-> netlist viewers ->RTL viewer.
17. Use an **appropriate radix** for readability – e.g. 4'd10 is the same as 4'b1010 but the former is usually more readable but this can depend on the context. Decimal is preferred unless individual bits are needed. This applies particularly to displaying signals in simulations.
18. Use **high level behavioural design** style where possible and avoid bit level instantiation of flip-flops and hand optimised logic gates without justified reasoning.

Appendix C: Programming a ROM IP-Block

To fully utilise our processor's instruction memory, we must be able to load it with compiled assembly programs. We will shortly add, in the skeleton files for this project, a Python script that will compile the assembly code into instruction words that are then written into a ".mif" file. Assuming your ROM block is setup correctly (see below), your "memory.mif" file will be programmed into the instruction memory of your processor when the project is uploaded to your DE board.

When creating the ROM ip block, you should select 'yes' when prompted about initialising the memory contents. Select 'browse' and then select the 'memory.mif' file from the skeleton code. If you wish to use any of the other examples, simply rename them to 'memory.mif', and they will automatically be used instead of changing this setting every time.



To compile some assembly code, place your assembly script into "assembly.2072" and run the Python script. **Be careful. This will overwrite whatever is currently in the 'memory.mif' file.**

If you do not feel comfortable writing your own assembly code, then you may use any of the provided scripts to demonstrate the functionality of your processor. Both the '.mif' and the raw assembly files for these examples are provided for your reference.

If you do wish to write your own assembly programs, then here is a list of the compilers (very limited) functionality:

- Empty lines will not be considered by the compiler
- When referencing registers, you must use the letter 'r' followed by the register number, for example, Register 0 must be referenced by '`r0`', Register 1 with '`r1`' and so on.

- You can write in-line comments using the "`//`" operator. The compiler will not consider anything written after the "`//`".
- You can separate the operands of an instruction with either a plain space, or with a comma followed by a space. i.e.

```
add r0, r2 // this is valid
sub r0 r2 // this is valid
mul r0,r2 // this is NOT valid
```

- Labels can be made by typing a word (with no spaces) followed by a colon (':').

```
this_is_a_label:
```

- These labels can then be referenced in a branch command by replacing the immediate value with the name of the label.

```
bez r0 this_is_a_label
```

There is some very limited error reporting built into the compiler, but if you have any questions or issues, post about them on EdStem. We also encourage you to post any interesting assembly code that you think other students may also like to use, this is fine to post publicly.